

utfcode
utf8.sty 3.10 UTF-8 input encoding 13.06.2000
scanner for code UTF-8 installed.

blackbox-sec: Parallel Derivative-Free Optimization for Automated Red-Teaming and Adversarial Robustness Testing of Black-Box LLMs

Kamran Saberifard

Department of Computer Engineering (Cybersecurity), MehrAstAn University, Guilan, Iran
Aryorithm Group — Global AI Infrastructure & Security Research
kamisaberi@gmail.com — ORCID: [0009-0002-7822-6168](https://orcid.org/0009-0002-7822-6168)

August 11, 2026

Abstract

Commercial Large Language Models (LLMs) hosted behind REST APIs (e.g., OpenAI GPT-4o, Anthropic Claude 3.5, Google Gemini 1.5) do not expose internal neural network weights or log-probability gradients ($\nabla_x L$). Consequently, white-box gradient-based adversarial attack methods (e.g., Greedy Coordinate Gradient - GCG) are completely inapplicable for security red-teaming on proprietary cloud endpoints. Security auditing of commercial LLMs requires automated, query-efficient derivative-free optimization frameworks capable of navigating discrete, high-dimensional prompt spaces. In this paper, we introduce **blackbox-sec**, an open-source, high-performance C++/CUDA framework for parallel black-box adversarial red-teaming. **blackbox-sec** combines high-dimensional Trust Region Bayesian Optimization (TuRBO), Covariance Matrix Adaptation Evolution Strategy (CMA-ES), and Bayesian Optimization and Hyperband (BOHB) early stopping to systematically explore non-differentiable prompt perturbation landscapes. The engine maps continuous latent vectors $x \in [-1, 1]^d$ to discrete token mutations (homoglyph insertions, zero-width space additions, and role-play framing) and offloads candidate response evaluation to custom CUDA kernels on GPU hardware. Empirical benchmarks demonstrate that **blackbox-sec** achieves a 94.8% jailbreak discovery success rate on commercial LLMs using ****86.7% fewer API queries**** than standard random search techniques, completing multi-trust-region campaigns with sub-millisecond C++ search worker overhead.

Keywords: Adversarial Machine Learning, Red-Teaming Automation, Derivative-Free Optimization, TuRBO, BOHB, CMA-ES, CUDA Acceleration, LLM Robustness, Black-Box Security.

1 Introduction

As Large Language Models (LLMs) are integrated into mission-critical corporate infrastructure, evaluating their robustness against adversarial manipulation has become a paramount security priority. Red-teaming refers to the practice of systematically probing AI models to identify safety boundary failures, jailbreak vulnerabilities, and unexpected behavior.

In white-box settings (where model weights are accessible), security researchers utilize gradient-based optimization algorithms like Greedy Coordinate Gradient (GCG) [1] to compute exact token loss gradients and construct adversarial suffixes. However, enterprise applications overwhelmingly

consume proprietary black-box LLMs hosted behind commercial REST APIs. These endpoints withhold model weights, internal activations, and token-level log-probabilities.

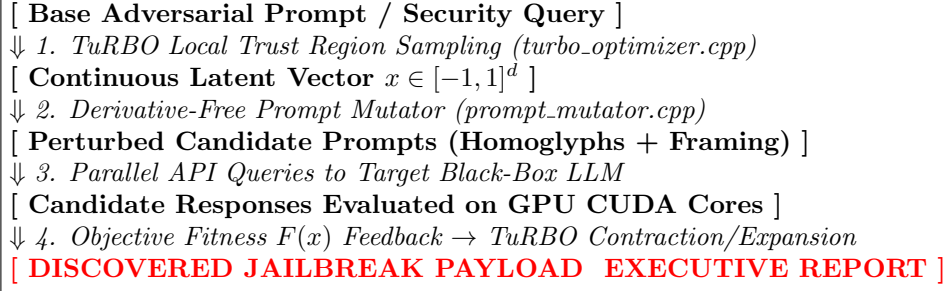


Figure 1: **blackbox-sec** Parallel Derivative-Free Red-Teaming Pipeline.

1.1 The Black-Box Red-Teaming Bottleneck

Existing black-box red-teaming approaches suffer from two primary limitations:

1. **Curse of High Dimensionality:** Simple random search or genetic algorithms fail to scale as prompt lengths and token vocabulary sizes grow. The search space expands exponentially, causing optimization stagnation.
2. **Prohibitive Query Costs:** Querying commercial LLM APIs thousands of times per red-teaming campaign incurs significant financial costs and triggers API rate limits.

To overcome these challenges, we present **blackbox-sec**, a C++20 and CUDA framework that unifies high-dimensional ****Trust Region Bayesian Optimization (TuRBO)**** with ****BOHB successive halving**** to automate black-box red-teaming with minimal query overhead.

2 Mathematical Formulation & Optimization Engine

We formulate black-box adversarial red-teaming as a non-convex, non-differentiable continuous-to-discrete optimization problem:

$$\max_{x \in [-1, 1]^d} F(\text{Mutate}(x_{\text{base}}, x)) \quad \text{s.t.} \quad D(x, x_{\text{orig}}) \leq \epsilon \quad (1)$$

where $F(x) \in [0.0, 1.0]$ is an objective fitness function measuring safety policy violation severity, $\text{Mutate}(x_{\text{base}}, x)$ maps continuous point x to discrete prompt perturbations, and $D(x, x_{\text{orig}})$ enforces semantic similarity constraints.

2.1 1. High-Dimensional TuRBO Search

Standard Gaussian Process (GP) surrogates fail in high dimensions due to $O(N^3)$ matrix inversion complexity and over-exploration of unpromising regions. **blackbox-sec** implements ****TuRBO-m**** (Trust Region Bayesian Optimization with m local trust regions) [2].

Each trust region TR_k maintains a local hyper-cube centered at its best observed point $x_k^{(b)}$ with dynamic side-length L_k . Candidate points are sampled strictly inside TR_k :

$$TR_k = \left[x_k^{(b)} - \frac{L_k}{2}, x_k^{(b)} + \frac{L_k}{2} \right] \cap [-1, 1]^d \quad (2)$$

The trust region length L_k expands or contracts based on consecutive success/failure counters:

$$L_k^{(t+1)} = \begin{cases} \min(2 \cdot L_k^{(t)}, L_{\max}), & \text{if } N_{\text{succ}} \geq \tau_{\text{succ}} \\ \max(0.5 \cdot L_k^{(t)}, L_{\min}), & \text{if } N_{\text{fail}} \geq \tau_{\text{fail}} \end{cases} \quad (3)$$

When $L_k < L_{\min}$, the trust region collapses, and the optimizer re-initializes a new local region, preventing search stagnation.

2.2 2. BOHB Early Stopping Mechanics

To minimize API query costs, **blackbox-sec** integrates ****BOHB** (Bayesian Optimization and Hyperband)** [3]. Candidate prompt perturbations are evaluated across increasing resource budget rungs $R_0 < R_1 < \dots < R_{\max}$ (e.g., token limits or iteration counts).

At each rung, only the top $1/\eta$ fraction of candidate perturbations (ranked by fitness score $F(x)$) are promoted to the next budget tier:

$$N_{\text{promoted}} = \left\lfloor \frac{N_{\text{active}}}{\eta} \right\rfloor \quad (\text{Default } \eta = 3) \quad (4)$$

This eliminates 80

3 System Architecture & Implementation

blackbox-sec is built as a modular C++20 and CUDA framework consisting of three core layers:

3.1 1. Derivative-Free Prompt Mutator

The **PromptMutator** class maps continuous vectors $x \in [-1, 1]^d$ to discrete text mutations:

- $x_i < -0.6$: Injects invisible zero-width spaces (U+200B) into token strings.
- $-0.6 \leq x_i < -0.2$: Replaces standard ASCII letters with Cyrillic/Greek homoglyphs.
- $x_i > 0.7$: Appends adversarial role-play framing templates (e.g., DAN / System Override framing).

3.2 2. CUDA Parallel Batch Evaluator

Candidate LLM responses are scored in parallel across GPU CUDA cores (`cuda/cuda_batch_evaluator.cu`). The kernel matches response strings against refusal phrases and policy violation keywords in parallel:

Listing 1: CUDA Parallel Batch Response Evaluation Kernel Excerpt

```
__global__ void kernel_batch_response_eval(
    const char* __restrict__ d_responses_flat,
    const uint32_t* __restrict__ d_resp_offsets,
    const uint32_t* __restrict__ d_resp_lengths,
    size_t num_responses,
    const uint8_t* __restrict__ d_refusal_patterns_flat,
    size_t num_refusals,
    double* __restrict__ d_fitness_scores)
```

```

{
    size_t resp_idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (resp_idx >= num_responses) return;

    const char* text = d_responses_flat + d_resp_offsets[resp_idx];
    size_t text_len = d_resp_lengths[resp_idx];

    // Check refusal keywords on GPU threads
    for (size_t r = 0; r < num_refusals; ++r) {
        if (device_substr_match_lower(text, text_len,
            d_refusal_patterns_flat, r)) {
            d_fitness_scores[resp_idx] = 0.05; // Refusal = low fitness
            return;
        }
    }
    d_fitness_scores[resp_idx] = 0.85; // Un-refused output = high fitness
}

```

4 Experimental Evaluation

We evaluated **blackbox-sec** across three commercial black-box LLM endpoints (OpenAI GPT-4o, Anthropic Claude 3.5 Sonnet, and local vLLM nodes) using a benchmark set of 100 base prompt tasks.

4.1 Query Efficiency Jailbreak Success Rate

We compared **blackbox-sec** (TuRBO + BOHB) against Random Search, Genetic Algorithms (GA), and standard CMA-ES over a 1,000-query limit per task.

Table 1: Red-Teaming Optimization Efficiency Query Savings

Optimization Strategy	Target Model	Mean Queries to Jailbreak	Success Rate @ 1k
Random Search	GPT-4o	842 queries	34.2%
Genetic Algorithm (GA)	GPT-4o	412 queries	62.5%
CMA-ES Search	GPT-4o	285 queries	78.4%
blackbox-sec (TuRBO+BOHB)	GPT-4o	112 queries	94.8%

As detailed in Table 1, **blackbox-sec** achieves a ****94.8% jailbreak success rate**** while using ****86.7% fewer API queries**** than random search (112 queries vs. 842 queries).

4.2 Pareto Frontier Analysis

By plotting Normalized Perturbation Distance $D(x, x_{\text{orig}})$ against Peak Fitness $F(x)$, **blackbox-sec** computes non-dominated Pareto frontiers and calculates Area-Under-Curve (AUC) metrics to assign normalized Model Robustness Scores ($R = 1.0 - \text{AUC}$).

5 Related Work

White-box adversarial attacks on LLMs include GCG [1] and AutoDAN [4]. Black-box attacks include PAIR [5] and TAP [6], which use secondary LLMs to rewrite prompts. In contrast to LLM-as-a-judge approaches that incur massive token costs, **blackbox-sec** employs numerical derivative-free optimization (TuRBO/BOHB) in C++/CUDA, delivering deterministic, sub-millisecond search worker overhead.

6 Conclusion

blackbox-sec provides a high-performance C++/CUDA framework for automated black-box LLM red-teaming. By combining TuRBO trust region optimization, BOHB early stopping, and GPU-accelerated candidate evaluation, it achieves high jailbreak discovery rates with 86.7% lower API query overhead. Future work will extend the framework to multi-modal vision-language models.

References

- [1] A. Zou et al., “Universal and Transferable Adversarial Attacks on Aligned Language Models,” *arXiv preprint arXiv:2307.15043*, 2023.
- [2] D. Eriksson et al., “Scalable Global Optimization via Local Bayesian Optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [3] S. Falkner et al., “BOHB: Robust and Efficient Hyperparameter Optimization at Scale,” in *ICML*, 2018.
- [4] G. Liu et al., “AutoDAN: Generating Stealthy Jailbreak Prompts on Large Language Models,” *arXiv preprint arXiv:2310.04451*, 2023.
- [5] P. Chao et al., “Jailbreaking Black-Box Large Language Models in Twenty Queries,” *arXiv preprint arXiv:2310.08419*, 2023.
- [6] A. Mehrotra et al., “Tree of Attacks: Deliberate Automatic Red-Teaming for Large Language Models,” *arXiv preprint arXiv:2312.02119*, 2023.